Environments

Tier: Free, Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

A GitLab environment represents a specific deployment target for your application, like development, staging, or production. Use it to manage different configurations and deploy code during various stages of your software lifecycle.

With environments, you:

- Keep your deployment process consistent and repeatable
- Track what code is deployed where
- Roll back to previous versions when problems occur
- Protect sensitive environments from unauthorized changes
- Control deployment variables per environment to maintain security boundaries
- Monitor environment health and get alerts if something goes wrong

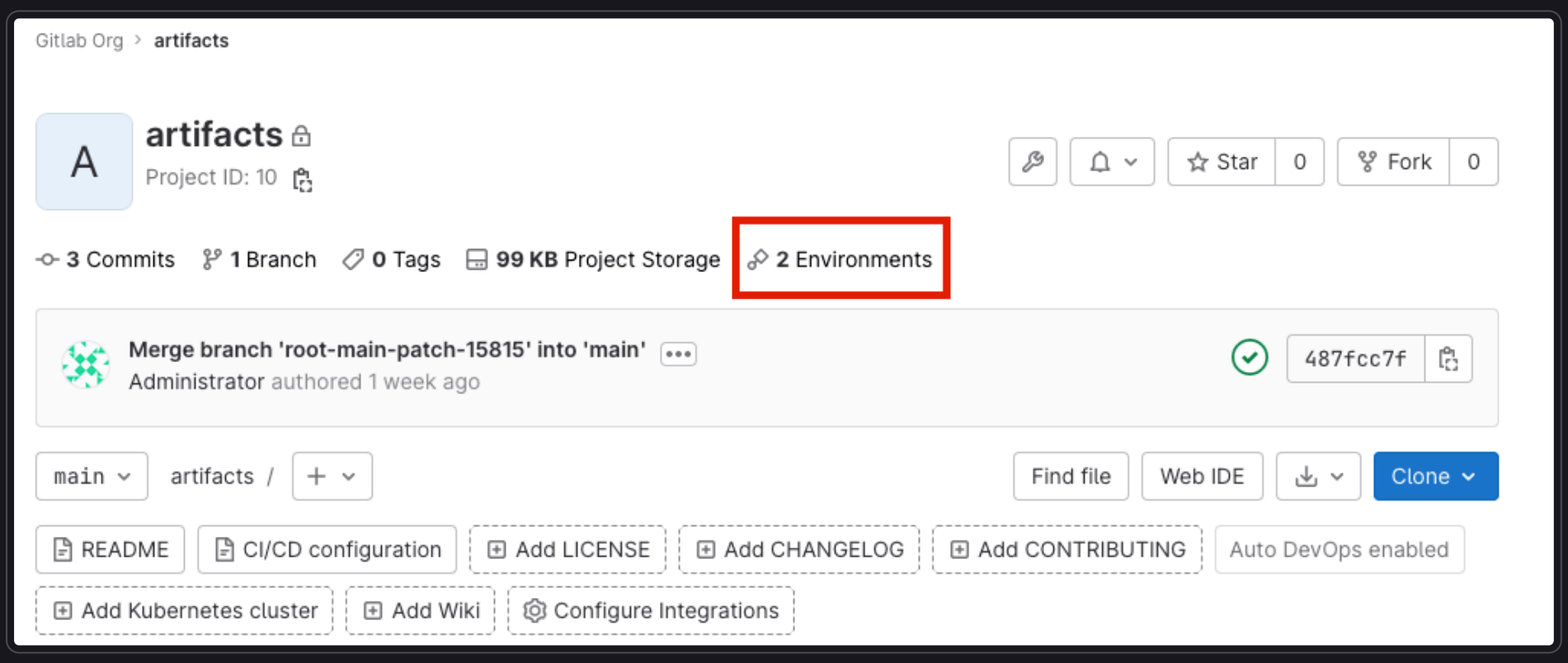
View environments and deployments

Prerequisites:

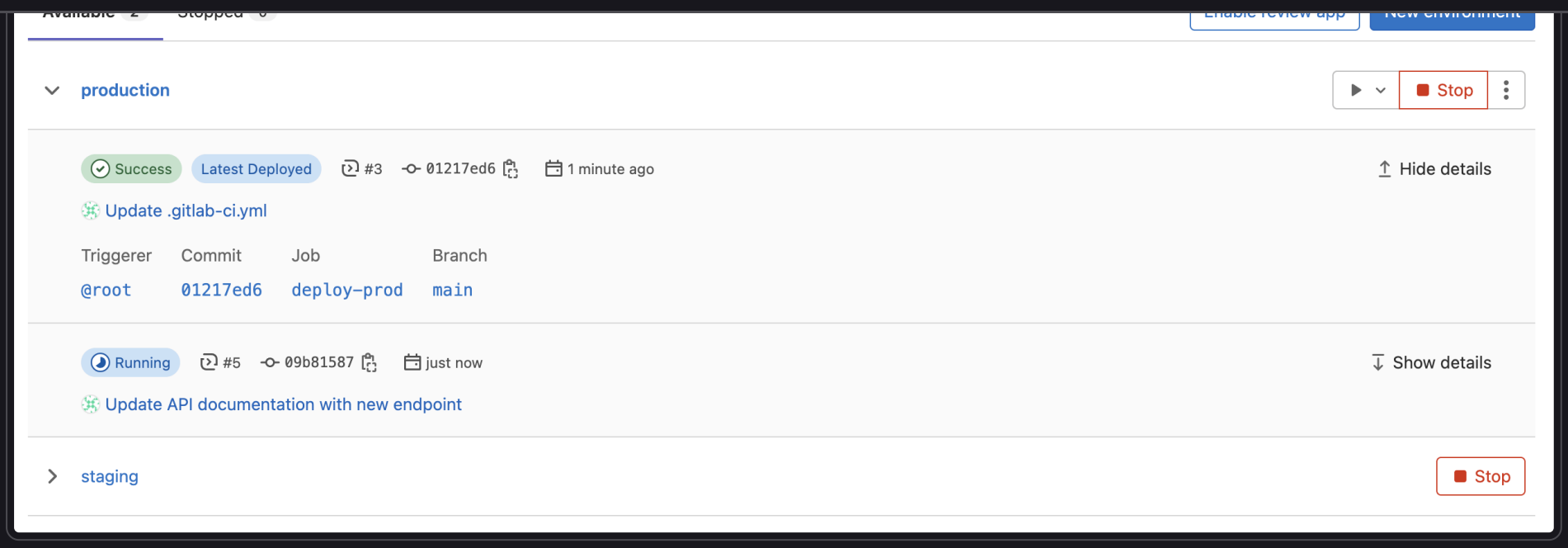
- In a private project, you must have the Reporter, Developer, Maintainer, or Owner role. See Environment permissions.

There are a few ways to view a list of environments for a given project:

- On the project's overview page, if at least one environment is available (that is, not stopped).

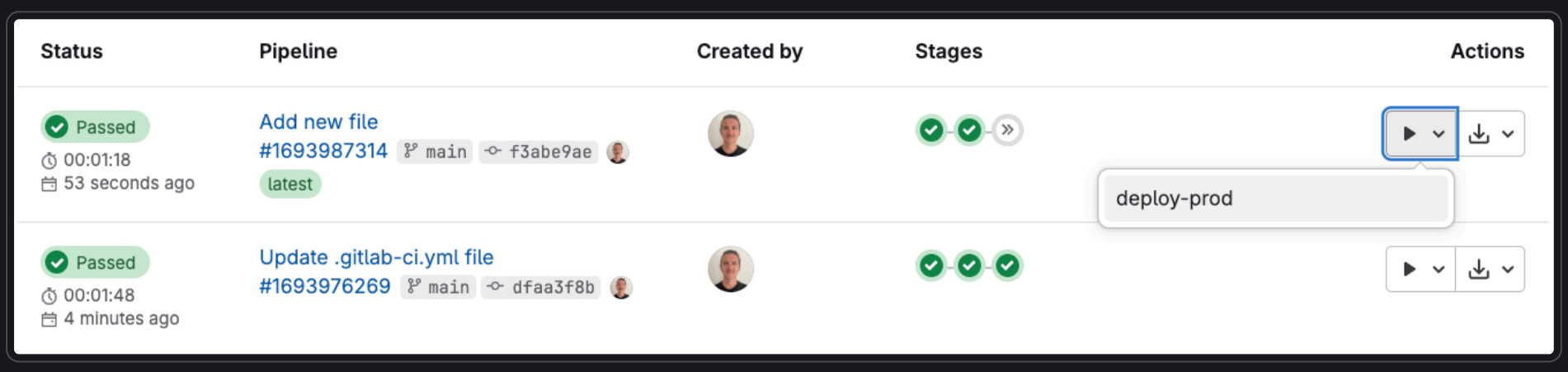


- In the left sidebar, select **Operate** > **Environments**. The environments are displayed.



- To view a list of deployments for an environment, select the environment name, for example, `staging`. Deployments show up in this list only after a deployment job has created them.

- To view a list of all manual jobs in a deployment pipeline, select the **Run** (▶) dropdown list.

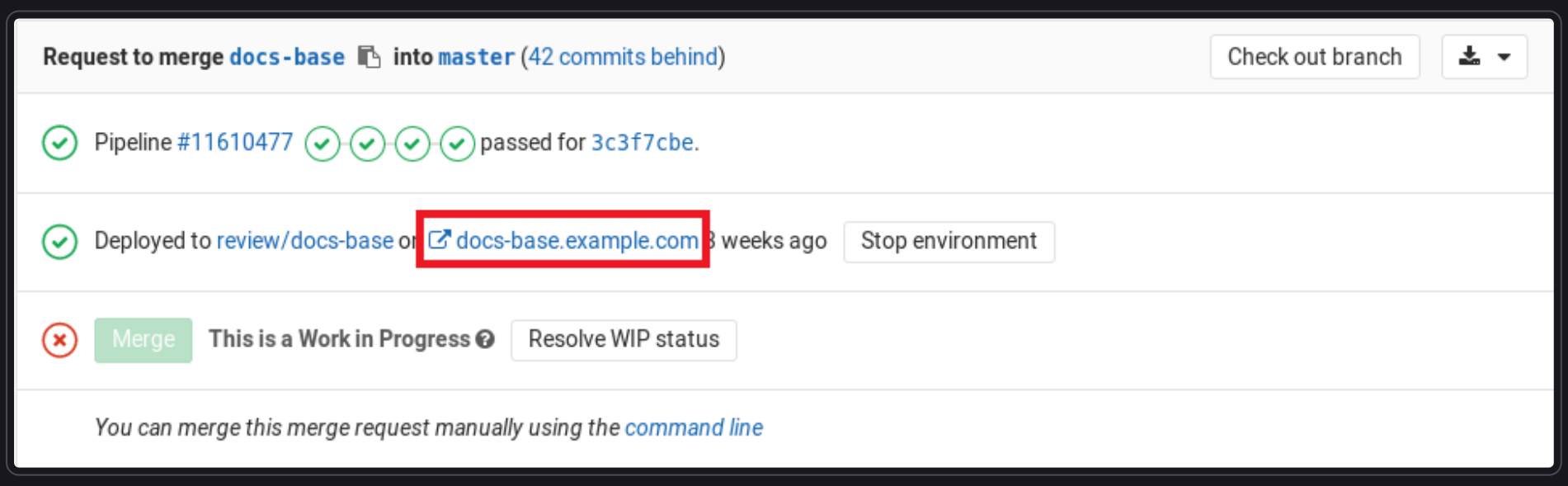


Environment URL

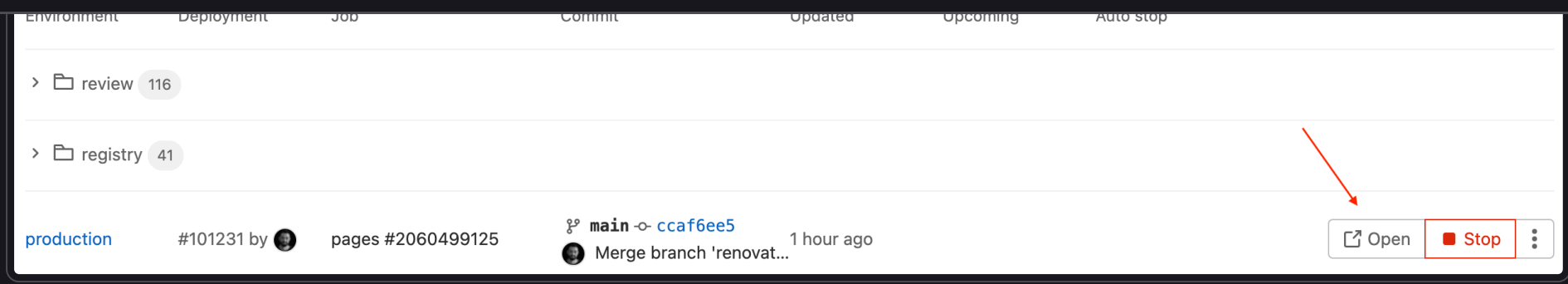
History >

The environment URL is displayed in a few places in GitLab:

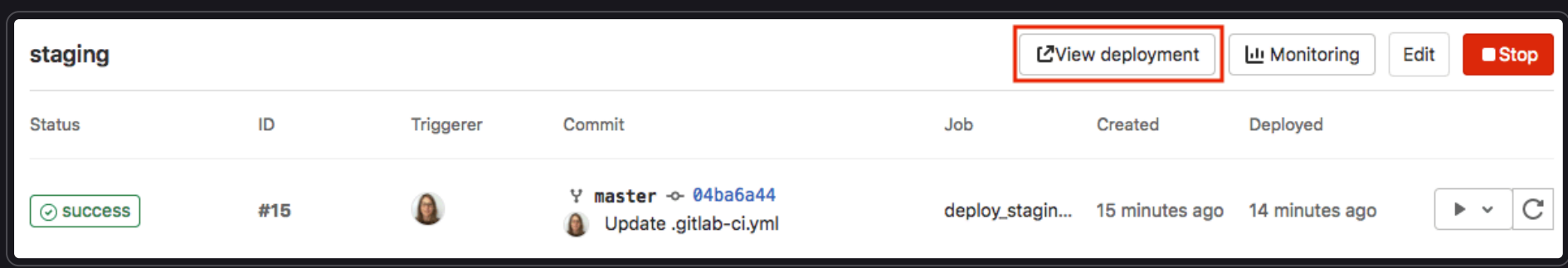
- In a merge request as a link:



- In the Environments view as a button:



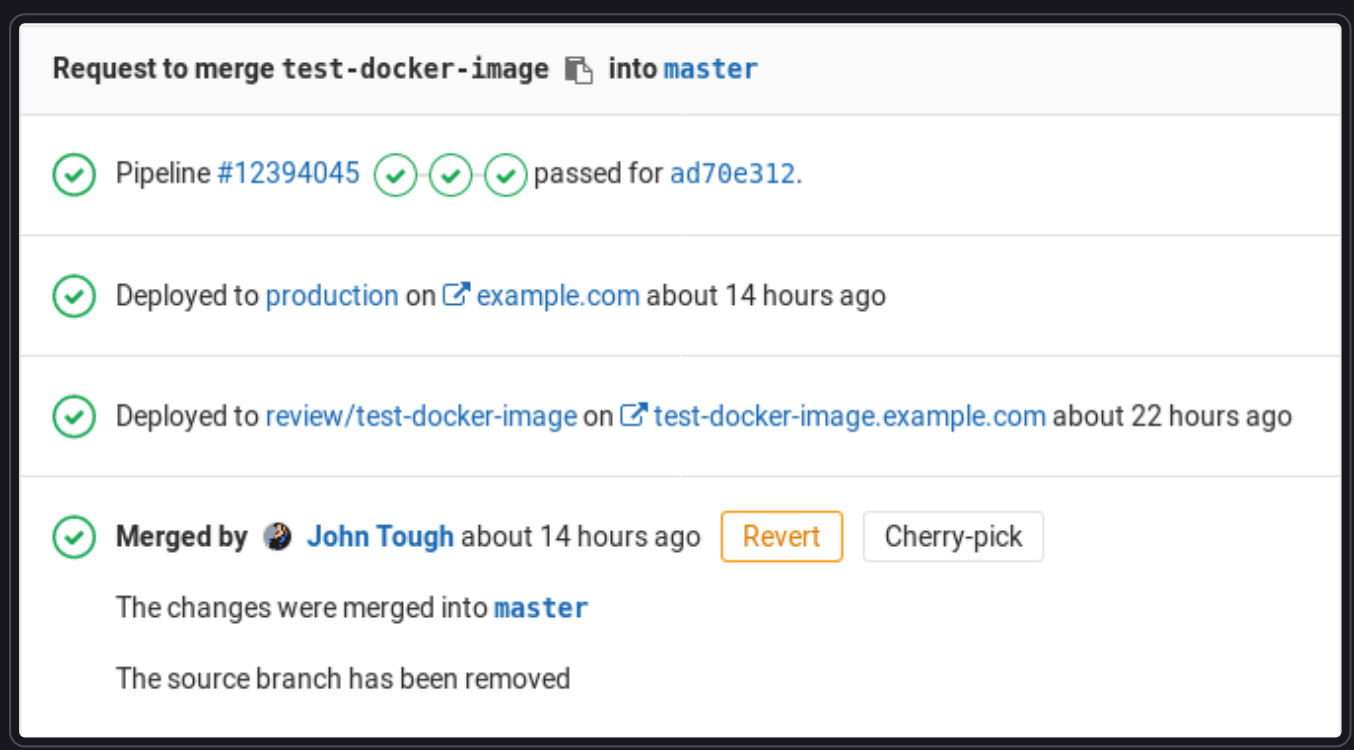
- In the Deployments view as a button:



You can see this information in a merge request if:

- The merge request is eventually merged to the default branch (usually `main`).
- That branch also deploys to an environment (for example, `staging` or `production`).

For example:



Go from source files to public pages

With GitLab Route Maps, you can go directly from source files to public pages in the environment set for review apps.

Types of environments

An environment is either static or dynamic.

Static environments:

- Are usually reused by successive deployments.
- Have static names. For example, `staging` or `production`.
- Are created manually or as part of a CI/CD pipeline.

Dynamic environments:

- Are usually created in a CI/CD pipeline and are used by only a single deployment, then either stopped or deleted.
- Have dynamic names, usually based on the value of a CI/CD variable.
- Are a feature of review apps.

- `stopping`: The *on stop job* has started. This state does not apply when there is no on stop job defined.
- `stopped`: Either the *on stop job* has run, or a user manually stopped the job.

Create a static environment

You can create a static environment in the UI or in your `.gitlab-ci.yml` file.

In the UI

Prerequisites:

- You must have the Developer, Maintainer, or Owner role.

To create a static environment in the UI:

1. In the top bar, select **Search or go to** and find your project.
2. In the left sidebar, select **Operate** > **Environments**.
3. Select **Create an environment**.
4. Complete the fields.
5. Select **Save**.

In your `.gitlab-ci.yml` file

Prerequisites:

- You must have the Developer, Maintainer, or Owner role.

To create a static environment, in your `.gitlab-ci.yml` file:

1. Define a job in the `deploy` stage.
2. In the job, define the environment `name` and `url`. If an environment of that name doesn't exist when the pipeline runs, it is created.

> Some characters cannot be used in environment names. For more information about the `environment` keywords, see the `.gitlab-ci.yml` keyword reference.

For example, to create an environment named `staging`, with URL `https://staging.example.com`:

```yaml
deploy_staging:
  stage: deploy
  script:
    - echo "Deploy to staging server"
  environment:
    name: staging
    url: https://staging.example.com
```

Create a dynamic environment

To create a dynamic environment, you use CI/CD variables that are unique to each pipeline.

Prerequisites:

- You must have the Developer, Maintainer, or Owner role.

2. In the job, define the following environment attributes:
   - `name`: Use a related CI/CD variable like `$CI_COMMIT_REF_SLUG`. Optionally, add a static prefix to the environment's name, which groups in the UI all environments with the same prefix.
   - `url`: Optional. Prefix the hostname with a related CI/CD variable like `$CI_ENVIRONMENT_SLUG`.

> Some characters cannot be used in environment names. For more information about the `environment` keywords, see the `.gitlab-ci.yml` keyword reference.

In the following example, every time the `deploy_review_app` job runs the environment's name and URL are defined using unique values.

```yaml
deploy_review_app:
  stage: deploy
  script: make deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_ENVIRONMENT_SLUG.example.com
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: never
    - if: $CI_COMMIT_BRANCH
```

Set a dynamic environment URL

Some external hosting platforms generate a random URL for each deployment, for example `https://94dd65b.amazonaws.com/qa-lambda-1234567`. This makes it difficult to reference the URL in the `.gitlab-ci.yml` file.

You can configure a deployment job to capture the generated URL as a dotenv variable and pass it to `environment:url`. Specify `artifacts:reports:dotenv` in your job. When the job finishes, GitLab parses the dotenv report and expands `environment:url` with the variable value. The assigned URL is then visible in the UI.

You can also combine a static prefix with the variable, for example `https://$DYNAMIC_ENVIRONMENT_URL`. If `DYNAMIC_ENVIRONMENT_URL` is `example.com`, the result is `https://example.com`.

For an overview, see set dynamic URLs after a job finished.

In the following example a review app creates a new environment for each merge request:

- The `review` job is triggered by every push, and creates or updates an environment named `review/your-branch-name`. The environment URL is set to `$DYNAMIC_ENVIRONMENT_URL`.
- When the `review` job finishes, GitLab updates the `review/your-branch-name` environment's URL. It parses the `deploy.env` report, extracts the variables, and uses them to expand and set the `environment:url`.

```yaml
```

```
environment URL.
    - echo "DYNAMIC_ENVIRONMENT_URL=$DYNAMIC_ENVIRONMENT_URL" >> deploy.env    # Add the value to a dotenv
file.
  artifacts:
    reports:
      dotenv: deploy.env                                                      # Report back dotenv file to
rails.
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: $DYNAMIC_ENVIRONMENT_URL                                             # and set the variable
produced in script to `environment:url`
    on_stop: stop_review

stop_review:
  script:
    - ./teardown-environment
  when: manual
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
```

Note the following:

- `stop_review` doesn't generate a dotenv report artifact, so it doesn't recognize the `DYNAMIC_ENVIRONMENT_URL` environment variable. Therefore you shouldn't set `environment:url` in the `stop_review` job.
- If the environment URL isn't valid (for example, the URL is malformed), the system doesn't update the environment URL.
- If the script that runs in `stop_review` exists only in your repository and therefore can't use `GIT_STRATEGY: none` or `GIT_STRATEGY: empty`, configure merge request pipelines for these jobs. This ensures that runners can fetch the repository even after a feature branch is deleted. For more information, see Ref Specs for Runners.

> For Windows runners, you should use the PowerShell `Add-Content` command to write to `.env` files.

```powershell
Add-Content -Path deploy.env -Value "DYNAMIC_ENVIRONMENT_URL=$DYNAMIC_ENVIRONMENT_URL"
```

Deployment tier of environments

Projects in the same group can use different environment names for the same deployment tier. For example, one project might use production while another uses custom-portal for the same tier. Group protected environments use deployment tiers to handle these differences.

The following deployment tiers are available:

- development
- testing
- staging
- production
- other

GitLab guesses deployment tiers from the environment name based on these patterns:

Ruby Regexp pattern	Deployment tier
`/(dev|review|trunk)/i`	development

`/(test|tst|int|ac(ce|)pt|qa|qc|control|quality)/i`	testing
`/(st(a|)g|mod(e|)l|pre|demo|non)/i`	staging
`/(pr(o|)d|live)/i`	production

Environment names that don't match any pattern are guessed as `other`.

To avoid the automatic guessing, use the `deployment_tier` keyword.

You cannot set deployment tiers in the UI.

Rename an environment

History >

You cannot rename an environment.

To achieve the same result as renaming an environment:

1. Stop the existing environment.
2. Delete the existing environment.
3. Create a new environment with the desired name.

CI/CD variables

To customize your environments and deployments, you can use any of the predefined CI/CD variables, and define custom CI/CD variables.

Limit the environment scope of a CI/CD variable

By default, all CI/CD variables are available to all jobs in a pipeline. If a test tool in a job becomes compromised, the tool could attempt to retrieve all CI/CD variables available to the job. To help mitigate this kind of supply chain attack, you should limit the environment scope of sensitive variables to only the jobs that require them.

Limit the environment scope of a CI/CD variable by defining which environments it can be available for. The default environment scope is the `*` wildcard, so any job can access the variable.

You can use specific matching to select a particular environment. For example, set the variable's environment scope to `production` to only allow jobs with an environment of `production` to access the variable.

You can also use wildcard matching (`*`) to select a particular environment group, like all review apps with `review/*`.

For example, with these four environments:

- `production`
- `staging`
- `review/feature-1`
- `review/feature-2`

These environment scopes match as follows:

↓ Scope / Environment →	`production`	`staging`	`review/feature-1`	`review/feature-2`
`*`	Match	Match	Match	Match

`production`	Match		
`staging`	Match		
`review/*`		Match	Match
`review/feature-1`		Match	

You should not use environment-scoped variables with `rules` or `include`. The variables might not be defined when GitLab validates the pipeline configuration at pipeline creation.

Search environments

History >

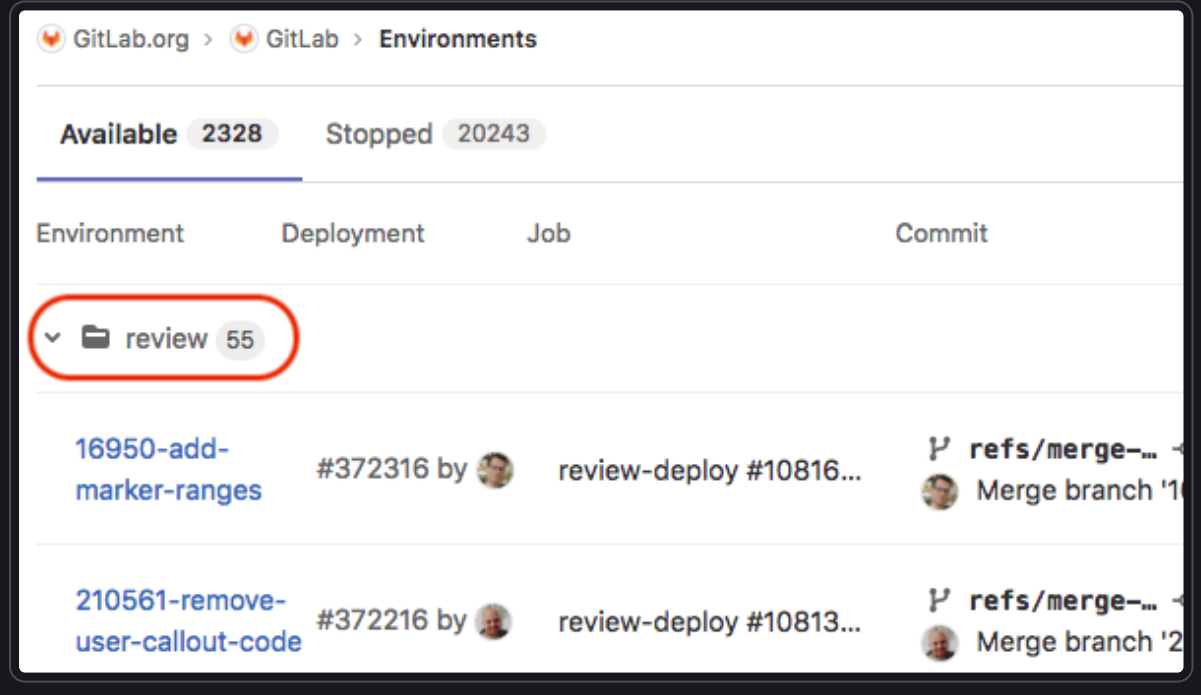
To search environments by name:

1. In the top bar, select **Search or go to** and find your project.
2. In the left sidebar, select **Operate** > **Environments**.
3. In the search bar, enter your search term.
   - The length of your **search term should be 3 or more characters**.
   - Matching applies from the beginning of the environment name.
     - For example, `devel` matches the environment name `development`, but `elop` does not.
   - For environments with a folder name format, matching applies after the base folder name.
     - For example when the name is `review/test-app`, search term `test` matches `review/test-app`.
     - Also searching with the folder name prefixed like `review/test` matches `review/test-app`.

Group similar environments

You can group environments into collapsible sections in the UI.

For example, if all of your environments start with the name `review`, then in the UI, the environments are grouped under that heading:



The following example shows how to start your environment names with `review`. The `$CI_COMMIT_REF_SLUG` variable is populated with the branch name at runtime:

yaml

```
    - echo "Deploy a review app"
  environment:
    name: review/$CI_COMMIT_REF_SLUG
```

Stopping an environment

Stopping an environment means its deployments are not accessible on the target server. You must stop an environment before it can be deleted.

When using the `on_stop` action to stop an environment, the job runs if it's not archived.

Stop an environment by using the UI

> To trigger an `on_stop` action and manually stop an environment from the Environments view, the stop and deploy jobs must be in the same `resource_group`.

To stop an environment in the GitLab UI:

1. In the top bar, select **Search or go to** and find your project.
2. In the left sidebar, select **Operate** > **Environments**.
3. Next to the environment you want to stop, select **Stop**.
4. On the confirmation dialog, select **Stop environment**.

Default stopping behavior

GitLab automatically stops environments when the associated branch is deleted or merged. This behavior persists even if no explicit `on_stop` CI/CD job is defined.

However, issue 428625 proposes to change this behavior so that production and staging environments stop only if an explicit `on_stop` CI/CD job is defined.

You can configure an environment's stopping behavior with the `auto_stop_setting` parameter in the Environments API.

Stop an environment when a branch is deleted

You can configure environments to stop when a branch is deleted.

In the following example, a `deploy_review` job calls a `stop_review` job to clean up and stop the environment.

- Both jobs must have the same `rules` or `only/except` configuration. Otherwise, the `stop_review` job might not be included in all pipelines that include the `deploy_review` job, and you cannot trigger `action: stop` to stop the environment automatically.
- The job with `action: stop` might not run if it's in a later stage than the job that started the environment.
- If you can't use merge request pipelines, set `GIT_STRATEGY` to `none` or `empty` in the `stop_review` job. Then, the runner doesn't try to check out the code after the branch is deleted.

yaml

```
    - echo "Deploy a review app"
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_ENVIRONMENT_SLUG.example.com
    on_stop: stop_review

stop_review:
  stage: deploy
  script:
    - echo "Remove review app"
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  when: manual
```

Stop an environment when a merge request is merged or closed

When you use the merge request pipelines configuration, the `stop` trigger is automatically enabled.

In the following example, the `deploy_review` job calls a `stop_review` job to clean up and stop the environment.

- When the **Pipelines must succeed** setting is turned on, you can configure the `allow_failure: true` keyword on the `stop_review` job to prevent it from blocking your pipelines and merge requests.

yaml

```yaml
deploy_review:
  stage: deploy
  script:
    - echo "Deploy a review app"
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    on_stop: stop_review
  rules:
    - if: $CI_MERGE_REQUEST_ID

stop_review:
  stage: deploy
  script:
    - echo "Remove review app"
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  rules:
    - if: $CI_MERGE_REQUEST_ID
      when: manual
```

When using this functionality together with merge trains, the `stop` job runs only if duplicate pipelines are avoided.

Stop an environment after a certain time period

You can set an environment to stop automatically after a certain time period.

Due to resource limitations, a background worker for stopping environments runs only once every hour. This means that environments may not be stopped after the exact time period specified, but are instead stopped when the background worker detects expired environments.

In the following example:

- Each commit on a merge request runs a `review_app` job that deploys the latest change to the environment and resets its expiry period.
- If the environment is inactive for more than a week, GitLab automatically runs the `stop_review_app` job to stop the environment.

```yaml
review_app:
  script: deploy-review-app
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    on_stop: stop_review_app
    auto_stop_in: 1 week
  rules:
    - if: $CI_MERGE_REQUEST_ID

stop_review_app:
  script: stop-review-app
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  rules:
    - if: $CI_MERGE_REQUEST_ID
      when: manual
```

The `environment:action` keyword can be used to reset the time that an environment is scheduled to stop. For more information, see Access an environment for preparation or verification purposes.

View an environment's scheduled stop date and time

When a environment has been scheduled to stop after a specified time period, you can view its expiration date and time.

To view an environment's expiration date and time:

1. In the top bar, select **Search or go to** and find your project.
2. In the left sidebar, select **Operate** > **Environments**.
3. Select the name of the environment.

The expiration date and time is displayed in the upper-left corner, next to the environment's name.

Override an environment's scheduled stop date and time

When a environment has been scheduled to stop after a specified time period, you can override its expiration.

To override an environment's expiration in the UI:

1. In the top bar, select **Search or go to** and find your project.
2. In the left sidebar, select **Operate** > **Environments**.
3. Select the environment name.
4. in the upper-right corner, select the thumbtack (📌).

To override an environment's expiration in the `.gitlab-ci.yml`:

1. Open the project's `.gitlab-ci.yml`.
2. Update the `auto_stop_in` setting of the corresponding deploy job to `auto_stop_in: never`.

The `auto_stop_in` setting is overridden and the environment remains active until it's stopped manually.

History >

Clean up stale environments when you want to stop old environments in a project.

Prerequisites:

- You must have the Maintainer or Owner role.

To clean up stale environments:

1. In the top bar, select **Search or go to** and find your project.
2. In the left sidebar, select **Operate** > **Environments**.
3. Select **Clean up environments**.
4. Select the date to use for determining which environments to consider stale.
5. Select **Clean up**.

Active environments that haven't been updated after the specified date are stopped. Protected environments are ignored and not stopped.

Run a pipeline job when environment is stopped

History >

You can define a stop job for the environment with an `on_stop` action in the environment's deploy job.

The stop jobs of finished deployments in the latest finished pipeline are run when an environment is stopped. A deployment or pipeline is finished if it has the successful, canceled, or failed status.

Prerequisites:

- Both the deploy and stop jobs must have the same rules or only/except configuration.
- The stop job must have the following keywords defined:
  - `when`, defined at either:
    - The job level.
    - In a rules clause. If you use `rules` and `when: manual`, you should also set `allow_failure: true` so the pipeline can complete even if the job doesn't run.
  - `environment:name`
  - `environment:action`

In the following example:

- A `review_app` job calls a `stop_review_app` job after the first job is finished.
- The `stop_review_app` is triggered based on what is defined under `when`. In this case, it is set to `manual`, so it needs a manual action from the GitLab UI to run.
- The `GIT_STRATEGY` is set to `none`. If the `stop_review_app` job is automatically triggered, the runner doesn't try to check out the code after the branch is deleted.

yaml

```
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_ENVIRONMENT_SLUG.example.com
    on_stop: stop_review_app

stop_review_app:
  stage: deploy
  variables:
    GIT_STRATEGY: none
  script: make delete-app
  when: manual
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
```

Multiple stop actions for an environment

History >

To configure multiple **parallel** stop actions on an environment, specify the `on_stop` keyword across multiple deployment jobs for the same `environment`, as defined in the `.gitlab-ci.yml` file.

When an environment is stopped, the matching `on_stop` actions from only successful deployment jobs are run in parallel, in no particular order.

> All `on_stop` actions for an environment must belong to the same pipeline. To use multiple `on_stop` actions in downstream pipelines, you must configure the environment actions in the parent pipeline. For more information, see downstream pipelines for deployments.

In the following example, for the `test` environment there are two deployment jobs:

- `deploy-to-cloud-a`
- `deploy-to-cloud-b`

When the environment is stopped, the system runs `on_stop` actions `teardown-cloud-a` and `teardown-cloud-b` in parallel.

yaml

```
    name: test
    on_stop: teardown-cloud-a

deploy-to-cloud-b:
  script: echo "Deploy to cloud b"
  environment:
    name: test
    on_stop: teardown-cloud-b

teardown-cloud-a:
  script: echo "Delete the resources in cloud a"
  environment:
    name: test
    action: stop
  when: manual

teardown-cloud-b:
  script: echo "Delete the resources in cloud b"
  environment:
    name: test
    action: stop
  when: manual
```

Stop an environment without running the `on_stop` action

There may be times when you want to stop an environment without running the defined `on_stop` action. For example, you want to delete many environments without using compute quota.

To stop an environment without running the defined `on_stop` action, execute the Stop an environment API with the parameter `force=true`.

Delete an environment

Delete an environment when you want to remove it and all its deployments.

Prerequisites:

- You must have the Developer, Maintainer, or Owner role.
- You must stop the environment before it can be deleted.

To delete an environment:

1. In the top bar, select **Search or go to** and find your project.
2. In the left sidebar, select **Operate** > **Environments**.
3. Select the **Stopped** tab.
4. Next to the environment you want to delete, select **Delete environment**.
5. On the confirmation dialog, select **Delete environment**.

Access an environment for preparation or verification purposes

History >

You can define a job that accesses an environment for various purposes, such as verification or preparation. This effectively bypasses deployment creation, so that you can adjust your CD workflow more accurately.

To do so, add either `action: prepare`, `action: verify`, or `action: access` to the `environment` section of your job:

```
stage: build
script:
  - echo "Building the app"
environment:
  name: staging
  action: prepare
  url: https://staging.example.com
```

This gives you access to environment-scoped variables, and can be used to protect builds from unauthorized access. Also, it's effective to avoid the prevent outdated deployment jobs feature.

If an environment is configured to stop after a certain time period, jobs with the `access` or `prepare` action will reset the scheduled stop time. The `environment:auto_stop_in` from the most recent successful deployment job to the environment is used when resetting the scheduled time. For example, if the most recent deployment used `auto_stop_in: 1 week` and is later accessed by a job with `action: access`, the environment will be rescheduled to stop one week from the completion of the accessing job.

To access an environment without changing the scheduled stop time, use the `verify` action.

Environment incident management

Production environments can go down unexpectedly, including for reasons outside of your control. For example, issues with external dependencies, infrastructure, or human error can cause major issues with an environment. Things like:

- A dependent cloud service goes down.
- A 3rd party library is updated and it's not compatible with your application.
- Someone performs a DDoS attack to a vulnerable endpoint in your server.
- An operator misconfigures infrastructure.
- A bug is introduced into the production application code.

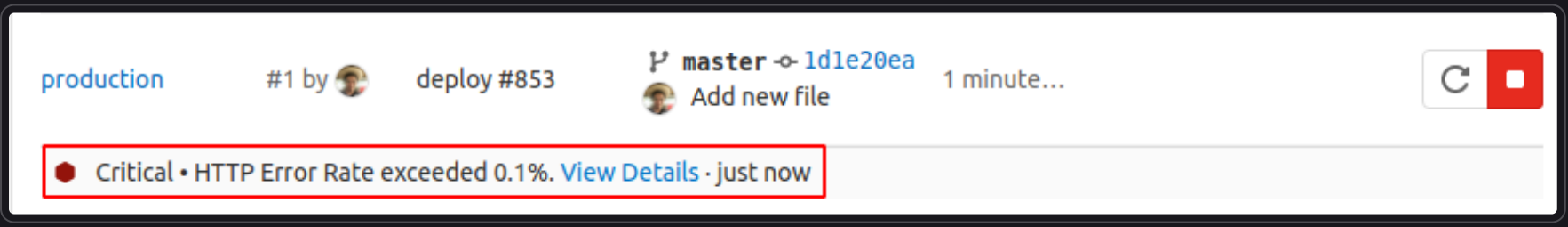
You can use incident management to get alerts when there are critical issues that need immediate attention.

View the latest alerts for environments

Tier: Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

If you set up an alert integration, alerts for environments are shown on the environments page. The alert with the highest severity is shown, so you can identify which environments need immediate attention.



When the issue that triggered the alert is resolved, it is removed and is no longer visible on the environments page.

If the alert requires a rollback, you can select the deployment tab from the environment page and select which deployment to roll back to.

Auto Rollback

Tier: Ultimate

In a typical Continuous Deployment workflow, the CI pipeline tests every commit before deploying to production. However, problematic code can still make it to production. For example, inefficient code that is logically correct can pass tests even though it causes severe performance degradation. Operators and SREs monitor the system to catch these problems as soon as possible. If they find a problematic deployment, they can roll back to a previous stable version.

GitLab Auto Rollback eases this workflow by automatically triggering a rollback when a critical alert is detected. For GitLab to select the appropriate environment for the rollback, the alert should contain a `gitlab_environment_name` key with the name of the environment. GitLab selects and redeploys the most recent successful deployment.

Limitations of GitLab Auto Rollback:

- The rollback is skipped if a deployment is running when the alert is detected.
- A rollback can happen only once in three minutes. If multiple alerts are detected at once, only one rollback is performed.

GitLab Auto Rollback is turned off by default. To turn it on:

1. In the top bar, select **Search or go to** and find your project.
2. In the left sidebar, select **Settings** > **CI/CD**.
3. Expand **Automatic deployment rollbacks**.
4. Select the checkbox for **Enable automatic rollbacks**.
5. Select **Save changes**.

Environment permissions

Depending on your role, you can interact with environments in public and private projects.

View environments

- In public projects, anyone can view a list of environments, including non-members.
- In private projects, you must have the Reporter, Developer, Maintainer, or Owner role to view a list of environments.

Create and update environments

- You must have the Developer, Maintainer, or Owner role to create a new environment, or update an existing unprotected environment.
- If an existing environment is protected and you don't have access to it, you cannot update the environment.

Stop and delete environments

- You must have the Developer, Maintainer, or Owner role to stop or delete an unprotected environment.
- If an environment is protected and you don't have access to it, you cannot stop or delete the environment.

Run deployment jobs in protected environments

If you can push or merge to the protected branch:

- You must have the Reporter, Developer, Maintainer, or Owner role.

If you can't push to the protected branch:

- You must be a part of a group with the Reporter role.

See Deployment-only access to protected environments.

This feature was deprecated in GitLab 14.5.

If you deploy to your environments with the help of a deployment service (for example, the Kubernetes integration), GitLab can open a terminal session to your environment. You can then debug issues without leaving your web browser.

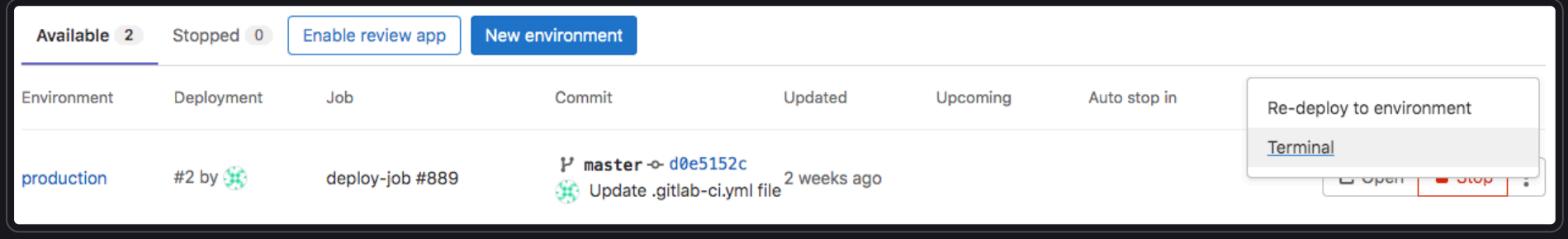
The Web terminal is a container-based deployment, which often lacks basic tools (like an editor), and can be stopped or restarted at any time. If this happens, you lose all your changes. Treat the Web terminal as a debugging tool, not a comprehensive online IDE.

Web terminals:

- Are available to project Maintainers and Owners only.
- Must be enabled.

In the UI, to view the Web terminal, either:

- From the **Actions** menu, select **Terminal**:



- On the page for a specific environment, on the right, select **Terminal** ( ).

Select the button to establish the terminal session. It works like any other terminal. You're in the container created by your deployment so you can:

- Run shell commands and get responses in real time.
- Check the logs.
- Try out configuration or code tweaks.

You can open multiple terminals to the same environment. They each get their own shell session and even a multiplexer like `screen` or `tmux`.

Related topics

- Dashboard for Kubernetes
- Deployments
- Protected environments
- Environments Dashboard
- Deployment safety

Troubleshooting

The job with `action: stop` doesn't run

In some cases, environments do not stop despite an `on_stop` job being configured. This happens when the job with the `action: stop` is not in a runnable state due to its `stages:` or `needs:` configuration.

For example:

- The environment might start in a stage that also has a job that failed. Then the jobs in later stages job don't start. If the job with the `action: stop` for the environment is also in a later stage, it can't start and the environment isn't deleted.
- The job with the `action: stop` might have a dependency on a job that has not yet completed.

```yaml
stages:
  - build
  - test
  - deploy

...

deploy_review:
  stage: deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_ENVIRONMENT_SLUG.example.com
    on_stop: stop_review

stop_review:
  stage: deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  when: manual
```

- Add a `needs` entry to the `action: stop` job so the job can start out of stage order:

```yaml
stages:
  - build
  - test
  - deploy
  - cleanup

...

deploy_review:
  stage: deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_ENVIRONMENT_SLUG.example.com
    on_stop: stop_review

stop_review:
  stage: cleanup
  needs:
    - deploy_review
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  when: manual
```

Try GitLab for free

Contribute to GitLab

Suggest updates

Help & Community

Resources

This job could not be executed because it would create an environment with an invalid parameter.

Get support

Post on the GitLab forum

Privacy statement

Use of generative AI

```
  script: echo
  environment: production/$ENVIRONMENT
```

Because the `$ENVIRONMENT` variable does not exist in the pipeline, GitLab tries to create an environment with a name `production/`, which is invalid in the environment name constraint.

To fix this, use one of the following solutions:

- Remove `environment` keyword from the deployment job. GitLab has already been ignoring the invalid keyword, therefore your deployment pipelines stay intact even after the keyword removal.
- Ensure the variable exists in the pipeline. Review the limitation on supported variables.
- If you have `environment:deployment_tier` in your `.gitlab-ci.yml`, ensure the value is one of the supported tiers: `production`, `staging`, `testing`, `development`, or `other`.

If you get this error on review apps

For example, if you have the following in your `.gitlab-ci.yml`:

```yaml
review:
  script: deploy review app
  environment: review/$CI_COMMIT_REF_NAME
```

When you create a new merge request with a branch name `bug-fix!`, the `review` job tries to create an environment with `review/bug-fix!`. However, the `!` is an invalid character for environments, so the deployment job fails because it was about to run without an environment.

To fix this, use one of the following solutions:

- Re-create your feature branch without the invalid characters, such as `bug-fix`.
- Replace the `CI_COMMIT_REF_NAME` predefined variable with `CI_COMMIT_REF_SLUG` which strips any invalid characters:

```yaml
review:
  script: deploy review app
  environment: review/$CI_COMMIT_REF_SLUG
```

Was this page helpful?